
task-app 30日間カリキュラム

Day 08

サイドバー付きのアプリレイアウトを作ろう

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	4
今日のゴール	4
なぜこれを作るのか	4
実装ステップ一覧	7
Step 1: providers.tsx を読む(tRPC クライアント設定・読む目安: 8分)	8
Step 2: ルートレイアウトのどこで Provider が囲んでいるか読む(読む目安: 5分)	11
Step 3: AppLayout を作る(サイドバーの骨格・読む目安: 15分)	14
Step 4: ダッシュボードに AppLayout を適用する(読む目安: 5分)	37
Step 5: ログインして全体の動作を確認する(読む目安: 5分)	39
Pro パターンで書こう(use client の影響範囲を最小化する)	44
完成コード全体	45
今日のまとめ	79
つまずきポイント	79
今日学んだ用語	81
応用課題: メニューの表示名とリンク先を見分ける	82
理解チェック	83
次回予告	84
次に読むもの	84
奥付	85

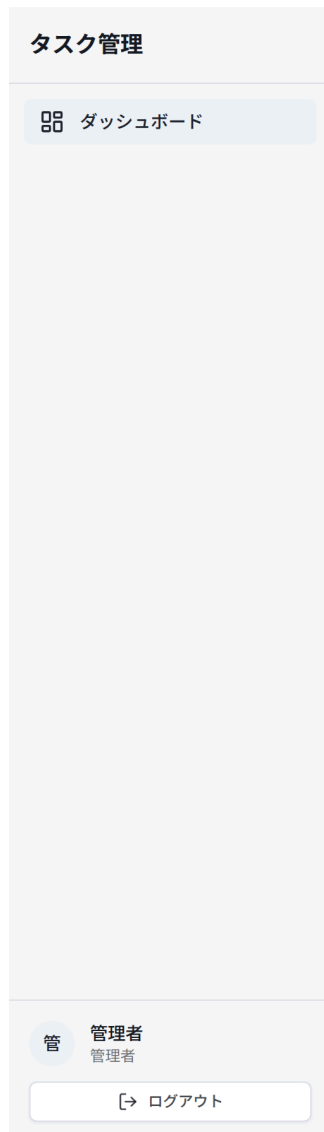


図1: Day 08 のサイドバー。上にロゴと「ダッシュボード」のメニュー、下にログイン中の管理者名とログアウトボタンが並んでいる

画像は今日のサイドバーです。すでにページがある「ダッシュボード」だけを表示します。「プロジェクト」は Day 09、「マイタスク」は Day 17 でページを作った直後に追加します。まだ無いページへのリンクを先に置くと、押した人が 404 ページへ進んでしまうためです。残りのメニューは Day 13 以降で画面を作るたびに1つずつ足していきます。

前回の振り返り

Day 07 で認証バックエンドを作りました。ログイン・登録・ログアウトが全部動くようになりました。

でも今のアプリ、ログインした先に何もありません。今日はサイドバー付きの共通レイアウトを自分の手で作って「アプリっぽい骨格」を完成させます。

今日のゴール

サイドバー付きのレイアウトを自分で作ってまずダッシュボードにこのレイアウトを適用します。Day 09 以降のページには作るたびに同じレイアウトを適用していきます。

- ☐ `src/app/providers.tsx` — 配布済みの tRPC Provider を読んで、どこまで届くか確認します。
- ☐ `src/component/layout/app-layout.tsx` — サイドバー + メインコンテンツ
- ☐ `src/app/dashboard/page.tsx` — AppLayout の中で動くダッシュボード
- ☐ ログアウトが動作することを確認します。

なぜこれを作るのか

Day 05～07 のログイン・登録画面は、配布済みの tRPC クライアントを通してすでに API を呼んでいます。今日はその設定がアプリのどこまで届くのかを確認します。そのうえで、ログイン後の各ページで使う共通のサイドバーレイアウトを作ります。

例え話: Day 07 で厨房(サーバー)を作りました。今日は客席のレイアウト(テーブル配置・通路・メニュー看板)を作ります。客が座ったときに「ちゃんとしたお店だな」と感じる骨格の部分です。

今日作る構造

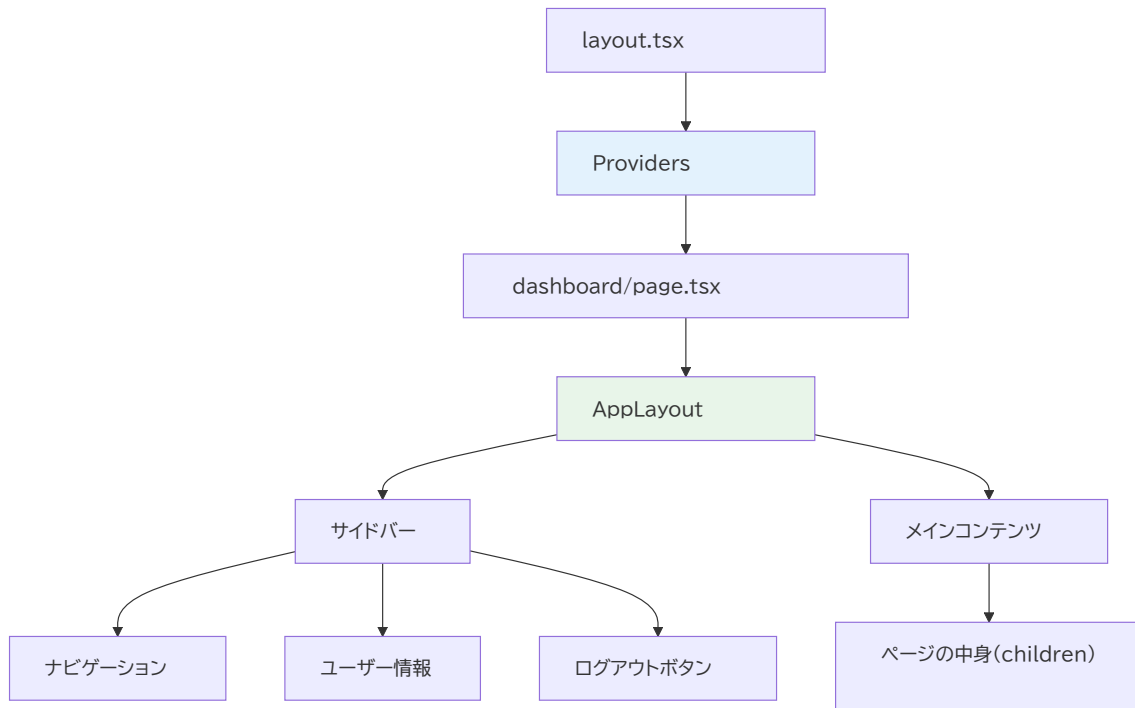


図 2: 今日作る構造

図は上から下へ、部品が別の部品を包む順番を表しています。いちばん外側の `layout.tsx` は全ページ共通の枠で、その内側に `Providers` を置くと下にぶら下がるページ全部から tRPC を呼び出せます。さらに内側の `AppLayout` がサイドバーとメインコンテンツに分かれ、サイドバーの中にナビゲーション・ユーザー情報・ログアウトボタンが並びます。

包む順番はそのまま「どこまで届くか」になります。仮に `Providers` をダッシュボードの内側だけに置くとログイン画面から tRPC を呼び出せません。

Day 08: サイドバー付きのアプリレイアウトを作ろう

やること / やらないこと

やること	やらないこと
配布済みの tRPC クライアント設定を読んで、フロントから API を呼べる理由を掴む	tRPC クライアント設定の自作 (scaffold で配布済み)
サイドバー + レイアウトを自分の手で書く	完成版の Sheet(画面端から開くパネル)式モバイルサイドバー(教材では作りません。Day 25 では別方式の横並びトップナビゲーションを作ります)
ログアウトを AlertDialog 付きで実装する	ユーザー編集機能

新しく学ぶ概念

概念	読み方	役割	例え
Provider	プロバイダー	コンポーネントツリー全体に値を配る仕組み	建物全体に電気を通す配電盤
use client (Day 02 の復習)	ユースクライアント	ブラウザ側で動く と宣言する	「この部品はお客さんの手元で動きます」
useQuery	ユースクエリ	サーバーからデータを取得する React Hook	注文票を厨房に出して結果を待つ

概念	読み方	役割	例え
useMutation	ユーズミュー テーション	サーバーのデータ を変更する React Hook	注文を厨房に送信する

実装ステップ一覧

ステップ	作業内容	読む時間の 目安	作成ファイル
Step 1	provide rs.tsx の 中身を読 む(tRPC + React Query)	8分	src/app/providers.tsx
Step 2	ルートレイ アウトのど こで Provider r が囲ん でいるか 読む	5分	src/app/layout.tsx 確認

Day 08: サイドバー付きのアプリレイアウトを作ろう

ステップ	作業内容	読む時間の 目安	作成ファイル
Step 3	AppLayout out を作る(サイド バーの骨 格)	15 分	<code>src/component/layout/app-layout.tsx</code>
Step 4	ダッシュ ボードに AppLayout out を適 用する	5 分	<code>src/app/dashboard/page.tsx</code>
Step 5	ログインし て全体の 動作を確 認する	5 分	なし

読む時間の合計(仮): 約 38 分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

Step 1: providers.tsx を読む(tRPC クライアント設定・読む目安: 8 分)

ゴール: 配布済みの設定を読んで、フロントエンドから tRPC API を呼べる仕組みを掴みます。

Day 07 で作った tRPC サーバーを、ブラウザ側から呼ぶには「クライアント」が必要です。scaffold は `src/trpc/` の設定ファイルと、それをアプリ全体に適用する Provider の両方を配布済みです。今日はどちらも書き足さず、中身を読んで仕組みを掴みます。



図3: Step 1: providers.tsx を読む

矢印はボタンを押してから画面が変わるまでの一往復です。ブラウザ側では `api.auth.login.mutate` のような関数呼び出しを書くだけで、`TRPCReactProvider` がそれを `/api/trpc` への HTTP リクエストに変換して送ります。返ってきた JSON も同じ道を逆にたどり、型の付いた値としてブラウザに戻ります。

この Provider が無いと `api` は送り先を知らないため呼んだ瞬間にエラーで止まります。

`src/app/providers.tsx` は Day 01 の scaffold が配布済みです。新しく作らず、開いて中身を確認します。

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
// filepath: src/app/providers.tsx
'use client';

import { Toaster } from 'react-hot-toast';
import { TRPCReactProvider } from '@trpc/react';

export function Providers({ children }: { children:
React.ReactNode }) {
  return (
    <TRPCReactProvider>
      {children}
    </TRPCReactProvider>
    <Toaster />
  );
}
```

コード	意味
'use client'	この Provider はブラウザ側で動く
TRPCReactProvider	scaffold が用意した tRPC + React Query の設定
Toaster	toast() で出す通知の表示場所。これが無いと通知は一切出ない
children	この下に置かれる全コンポーネントが tRPC を使える

scaffold の `src/trpc/react.tsx` の中身が気になったら開いてみてもいいです。QueryClient と httpBatchLink の設定が入っています。

確認ポイント:

- ☐ `src/app/providers.tsx` の中身を確認しました。
- ☐ `'use client'` が先頭にあります。
- ☐ `<Toaster />` が入っています。

Step 2: ルートレイアウトのどこで Provider が囲んでいるか読む(読む目安: 5分)

ゴール: アプリ全体で tRPC が使える理由を、ルートレイアウトを読んで掴みます。

`src/app/layout.tsx` を開きます。こちらも scaffold が配布済みで、Provider はすでに組み込まれています。書き換えずに、どこで囲んでいるかを確認します。

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
// filepath: src/app/layout.tsx
import type { Metadata } from 'next';
import { Inter, JetBrains_Mono, Noto_Sans_JP } from
'next/font/google';
import { Providers } from './providers';
import './globals.css';

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-inter',
  display: 'swap',
});
```

同じ書き方で `JetBrains_Mono` を `--font-jetbrains-mono`、
`Noto_Sans_JP` を `--font-noto-sans-jp` として読み込んでいます。続きは次のとおりです。

```
// filepath: src/app/layout.tsx(続き)
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="ja" className={` ${inter.variable} ${
jetBrainsMono.variable} ${notoSansJP.variable}`}>
      <body>
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}
```

確認する箇所	意味
<code>import { Providers }</code>	Step 1 で見た Provider を読み込む
<code><Providers>{children}</Providers></code>	全ページを Provider で囲む
<code>className={...}</code>	フォントの変数を全ページへ渡す

`--font-inter` などの変数名は `src/app/layout.tsx` にある `Inter({ variable: '--font-inter' })` などの `variable` 指定で決めています。Day 01 で書いた `globals.css` がこの変数を参照しているので消すとフォント指定が黙って効かなくなります。

Day 08: サイドバー付きのアプリレイアウトを作ろう

`<Providers>` で全体を囲んだのでアプリのどこからでも
`api.auth.login.useMutation()` のように tRPC を呼びます。

確認ポイント:

- ☐ `src/app/layout.tsx` に `<Providers>` が入っていることを確認しました。
- ☐ この時点で `npm run dev` してエラーが出ないことを確認

Step 3: AppLayout を作る(サイドバーの骨格・読む目安: 15分)

ゴール: サイドバー + メインコンテンツのレイアウトコンポーネントを作ります。

ここが今日のメインです。認証チェック、ナビゲーション、ログアウトを 1 つのレイアウトに組み込みます。

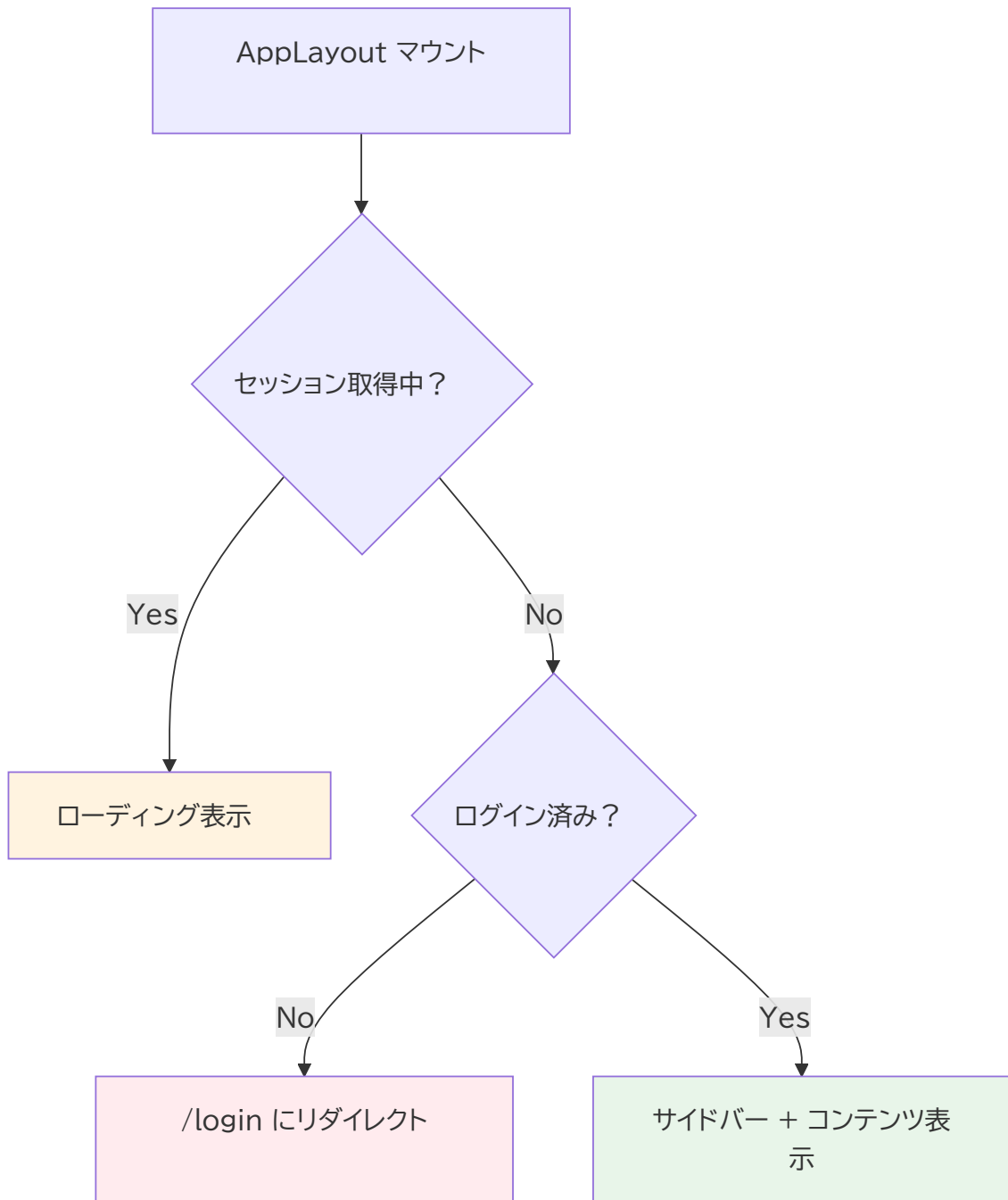


図4: Step 3: AppLayout を作る

この分岐はAppLayout が中身を描く前に必ず通る判定です。セッションの問い合わせが終わるまでは読み込み中の表示を出し、終わってからログイン済みかどうかを見ます。ログインしていなければ何も描かずに `/login` へ送るので未ログインの人にサイドバーが一瞬でも見えることはありません。

Day 08: サイドバー付きのアプリレイアウトを作ろう

この書き方は問い合わせ自体が失敗したときも未ログインとして扱います。ログイン済みの人が通信の失敗でログイン画面へ送られることがある、ということです。実務では `error` も受け取り、失敗したときは再試行の案内を出してから判断します。

この判定をレイアウト側に置いたおかげで、この先に作るページは自分で認証を確認せずに済みます。

`src/component/layout/app-layout.tsx` を新規作成します。

3-1. インポートと型定義

```
// filepath: src/component/layout/app-layout.tsx
'use client';

import { useQueryClient } from '@tanstack/react-query';
import { LayoutDashboard, Logout } from 'lucide-react';
import Link from 'next/link';
import { usePathname, useRouter } from 'next/navigation';
import { useEffect, useRef, useState } from 'react';
import {
  AlertDialog,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from '@component/ui/alert-dialog';
```

`import` はこのファイルで使う道具を先に並べる宣言です。`useQueryClient` は画面をまたいで保存された問い合わせ結果を操作します。`lucide-react` はサイドバーのアイコン、`Link` はページ移動、`usePathname` は今いるページの URL を取る道具で、現在のメニューを目立たせるときに使います。`useState` は画面の状態を覚えておく仕組みです。`useEffect` (画面が表示された後など、決まったタイミングで処理を走らせる React の仕組み)はあとで「ブラウザ側の準備が終わったか」を記録するときに使います。

`AlertDialog` で始まる 8 個はログアウト前の確認ダイアログを組み立てる部品一式です。`shadcn/ui` のダイアログは 1 つの部品ではなく、枠・見出し・本文・ボタンに分かれています。だから使う分だけ名前を並べて取り込みます。1 つでも取り込み漏れがあるとその名前を書いた行で「定義されていない」というエラーが出ます。

確認ポイント:

- ☐ ログアウト確認に使う `AlertDialog` 一式を `import` しています。

```
// filepath: src/component/layout/app-layout.tsx(続き)
import { Button } from '@component/ui/button';
import { cn } from '@lib/utils';
import { api } from '@trpc/react';
```

`cn` は条件によって変わるクラス名を 1 つの文字列にまとめる関数です。あとでメニューの現在地を色分けするとき選択中と未選択のクラスを混ぜて渡します。

Day 08: サイドバー付きのアプリレイアウトを作ろう

+ でつないでも動きます。ただ、空白を1つ入れ忘れると

`bg-gray-100text-blue-600` のようにくっついてそのクラスだけ効かなくなります。画面は崩れず、ただ色が付かないだけです。だから原因を探すのに時間がかかります。

`api` は tRPC のクライアントです。これを取り込まないとあとで書く

`api.auth.getSession` の行で「名前が見つからない」というエラー画面が出て、ページを表示できなくなります。

確認ポイント:

- ☐ ログアウト用の `AlertDialog` を import しています。
- ☐ tRPC 用の `api` を import しています。

```
// filepath: src/component/layout/app-layout.tsx(続き)
interface MenuItem {
  text: string;
  icon: React.ReactNode;
  path: string;
}

const menuItems: MenuItem[] = [
  {
    text: 'ダッシュボード',
    icon: <LayoutDashboard className="h-5 w-5" />,
    path: '/dashboard',
  },
];
```

コード	意味
<code>'use client'</code>	ユーザー操作(ナビゲーション、ログアウト)があるので Client Component
<code>lucide-react</code>	アイコンライブラリ(scaffold でインストール済み)
<code>AlertDialog</code>	ログアウト前の確認ダイアログ (scaffold の UI コンポーネント)
<code>menuItems</code>	サイドバーに表示するメニュー項目の 定義

3-2. コンポーネント本体(認証チェック + ログアウト)

```
// filepath: src/component/layout/app-layout.tsx(続き)
export function AppLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const [hasMounted, setHasMounted] = useState(false);
  const [logoutDialogOpen, setLogoutDialogOpen] =
    useState(false);
  const [logoutError, setLogoutError] = useState<string |
    null>(null);
  const logoutLockedRef = useRef(false);
  const queryClient = useQueryClient();
  const pathname = usePathname();
  const router = useRouter();

  const { data: session, isLoading } =
    api.auth.getSession.useQuery();
```

ログアウトの失敗を画面に残すため、ダイアログの開閉とエラー文を `useState` で覚えます。 `useRef` (再描画を待たずに値を保持する React の仕組み)は、連続クリックをその場で止めるために使います。

```
// filepath: src/component/layout/app-layout.tsx(続き)
const logoutMutation = api.auth.logout.useMutation({
  onSuccess: async () => {
    await queryClient.cancelQueries();
    queryClient.clear();
    router.push('/login');
    router.refresh();
    setLogoutDialogOpen(false);
  },
  onError: () => {
    setLogoutError(
      'ログアウトの完了を確認できませんでした。' +
      '通信状況を確認して、もう一度お試しください。');
  },
  onSettled: () => {
    logoutLockedRef.current = false;
  },
});

const handleLogout = () => {
  if (logoutLockedRef.current ||
logoutMutation.isPending) return;
  logoutLockedRef.current = true;
  setLogoutError(null);
  logoutMutation.mutate();
};
```

Day 08: サイドバー付きのアプリレイアウトを作ろう

`onSettled` (成功・失敗のどちらでも通信の最後に動く処理)で

`logoutLockedRef` のロックを解除します。失敗した後も解除されるため、案内を読んでから再試行できます。

```
// filepath: src/component/layout/app-layout.tsx(続き)
const openLogoutDialog = () => {
  setLogoutError(null);
  setLogoutDialogOpen(true);
};

const handleLogoutDialogOpenChange = (open: boolean) =>
{
  if (!open && (logoutLockedRef.current ||
logoutMutation.isPending)) return;
  if (open) setLogoutError(null);
  setLogoutDialogOpen(open);
};
```

状態と問い合わせを、関数の先頭でまとめて宣言しています。`useState` や

`useQuery` のようなフックは毎回同じ順番で呼ばれることを前提に作られています。`if` の中や `return` より後ろに置くと順番が変わり、React が値を取り違えてエラーになります。並べる場所には理由があります。

`logoutMutation` を関数の先頭で用意しておくのは `useMutation` もフックだからです。フックは毎回同じ順番で呼ばれる前提なので `onClick` の中には書けません。実際に通信が始まるのはボタンが押されて `mutate` が呼ばれたときです。

ログアウト API が成功した後の処理は、次の順番です。失敗した場合はキャッシュと画面を残し、ダイアログ内に再試行の案内を出します。

順番	処理
1	<code>cancelQueries()</code> で実行中の問い合わせを止める
2	<code>clear()</code> で、前の利用者の名前やタスクをブラウザ内のキャッシュ(取得済みデータを一時保存する場所)から消す
3	ログイン画面へ移動し、 <code>router.refresh()</code> でサーバー側の表示を作り直す

問い合わせの停止を待つため、`onSuccess` に `async` を付けて `await` を使います。停止と削除の順番が逆だと、消した直後に遅れて終わった問い合わせが前の利用者のデータを入れ直す可能性があります。

確認ポイント:

- ☐ `getSession` でログイン状態を取得しています。
- ☐ 実行中の問い合わせを止めてからキャッシュを消しています。
- ☐ ログアウト成功後に `/login` へ戻しています。

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
// filepath: src/component/layout/app-layout.tsx(続き)

useEffect(() => {
  setHasMounted(true);
}, []);

useEffect(() => {
  if (hasMounted && !isLoading && !session) {
    router.push('/login');
  }
}, [hasMounted, isLoading, session, router]);

if (!hasMounted || isLoading) {
  return (
    <div className="flex h-screen items-center justify-center">
      <p className="text-muted-foreground">
        読み込み中...
      </p>
    </div>
  );
}

if (!session?.user) {
  return null;
}
```

読み込み中の表示をここでは自分で書いています。Day 09 で別のページの読み込み中表示を作るときは、配布済みの `PageLoadingSpinner` を使います。今日の `AppLayout` は仕組みを追いやすいように素の要素のまま完成させます。

コード	意味	例え
<code>hasMounted</code>	SSR(サーバー側で先に HTML を組み立てる仕組み)と CSR(ブラウザ側で後から HTML を組み立てる仕組み)のズレ防止フラグ	店が開店してから初めて客をチェックする
<code>api.auth.getSession.useQuery()</code>	サーバーに「今ログインしているか」を問い合わせる	入口でリストバンドチェック
<code>logoutMutation</code>	ログアウト API を呼ぶ準備	退出手続きのボタン
<code>router.push('/login')</code>	ログイン画面へ飛ばす	受付に案内する

`hasMounted` が必要な理由: Next.js はサーバーで HTML を生成してからブラウザに送ります(SSR)。session Cookie には Day 07 で `httpOnly` を付けたのでブラウザ側の JavaScript からは中身を読めません。ログイン状態を知る道は `api.auth.getSession.useQuery()` でサーバーに聞くことだけです。このとき Cookie はリクエストへ自動で付いて飛び、中身を開いて誰

Day 08: サイドバー付きのアプリレイアウトを作ろう

なのかを判定するのはサーバー側です。答えが返ってくるのは画面が出たあとなのでこのフラグで「ブラウザ側の準備ができてから判定する」ようにしてチラつきを防ぎます。

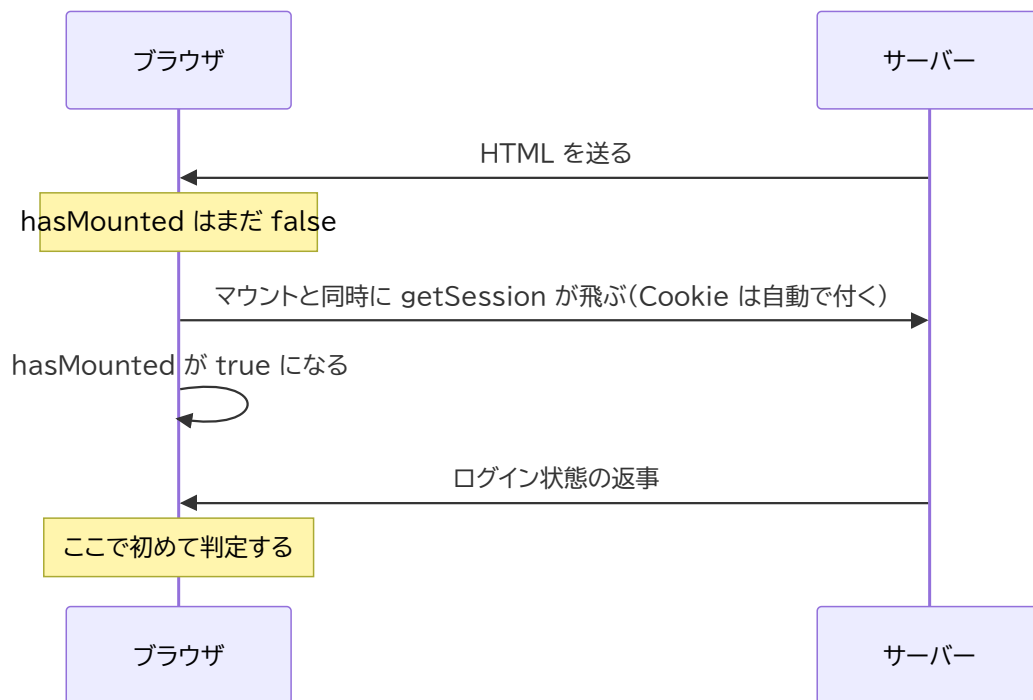


図 5: 3-2. コンポーネント本体(認証チェック + ログアウト)

時間の流れは上から下です。`useQuery` に `enabled` を渡していないので問い合わせはマウントと同時に始まります。応答が届く前のリダイレクトを防ぐのは `useEffect` 内の `!isLoading` です。

`hasMounted` を使うと初回は必ず読み込み中の表示になります。このフラグだけを外しても `!isLoading` があるため応答を待たずにログイン画面へ移動することはありません。

3-3. レイアウト JSX(サイドバー + コンテンツ)

```
// filepath: src/component/layout/app-layout.tsx(続き)
return (
  <div className="flex h-screen bg-background">
    {/* サイドバー */}
    <aside className="hidden w-64 flex-col border-r
bg-sidebar md:flex">
      {/* □ゴ */}
      <div className="border-b border-sidebar-border
p-4">
        <h1 className="text-lg font-bold
text-sidebar-foreground">
          タスク管理
        </h1>
      </div>
    </div>
  </div>
)
```

サイドバーを `<div>` ではなく `<aside>` で書いています。見た目はどちらも同じですが `<aside>` には「ここは本文ではなく脇の案内です」という意味が付きます。読み上げソフトを使う人はこの目印でページの主役がどこかを判断します。`<div>` で揃えてしまうとその手がかりが消えます。

`hidden` と `md:flex` を並べているのは画面が狭いときにサイドバーを隠すためです。`w-64` は幅 256px を指します。スマートフォンの画面にこの帯が居座ると本文に残る幅が 100px ほどになり、文章が1文字ずつ折り返されて読めなくなります。

Day 08: サイドバー付きのアプリレイアウトを作ろう

ここで先に断っておくことがあります。消えるのはサイドバーの帯ごとなので幅 768px 未満ではナビゲーションのリンクとログアウトボタンがまとめて画面から消えます。代わりのハンバーガーメニューを、このカリキュラムでは作りません。Day 04 で公開した URL をスマートフォンから開くとログインした先に移動する手段の無い画面が出ます。動作確認はパソコンのブラウザで進めてください。パソコンのブラウザでウィンドウ幅を 768px 以上に広げると帯が戻ります。自分の書き間違いではありません。

確認ポイント:

- ☐ 左側にサイドバーを作っています。
- ☐ サイドバー上部のロゴとして **タスク管理** を表示しています。
- ☐ 画面幅を 768px より狭くするとサイドバーごと消えることを確認しました（仕様どおりの動き）。

```

    { /* filepath:
src/component/layout/app-layout.tsx(続き) */}
    { /* ナビゲーション */}
    <nav className="flex-1 p-3">
      <ul className="space-y-1">
        {menuItems.map((item) => (
          <li key={item.path}>
            <Link
              href={item.path}
              className={cn(
                'flex items-center gap-3 rounded-md px-3 py-2 text-sm ' +
                  'transition-colors',
                pathname === item.path
                  ?
                'bg-sidebar-accent text-sidebar-accent-foreground' +
                  ' font-medium'
                  :
                'text-sidebar-foreground/70 hover:bg-sidebar-accent/50' +
                  ' hover:text-sidebar-foreground',
              )}

```

この前半でリンク先と、現在地に応じた色を決めています。続きでアイコンと文字を描き、一覧のタグを閉じます。2つのブロックは同じ `Link` の続きなので、間に別のコードは入れません。

```
        >
        { /* filepath:
src/component/layout/app-layout.tsx(続き) */
        {item.icon}
        {item.text}
        </Link>
      </li>
    )}
  </ul>
</nav>
```

メニューを `map` で組み立てるのは項目を増やすときに `menuItems` へ1行足すだけで済むようにするためです。今は1項目だけなので `<li>` を手で書いても動きます。ただし項目が増えるたびに同じクラス名を書き写すことになり、1つだけ直し忘れた項目がいずれ出ます。`key` に `item.path` を渡すのはReact が並び替えのときにどの項目が動いたかを見分けるためです。

`pathname === item.path` の比較で、今開いているページのリンクだけ背景色を変えています。この目印が無いとユーザーは自分がどこにいるかを画面から判断できません。移動したかどうかも分からなくなります。

確認ポイント:

- ☐ `menuItems.map` でメニューを表示しています。
- ☐ 現在ページは背景色で強調しています。

```

    { /* filepath:
src/component/layout/app-layout.tsx(続き) */ }
    { /* ユーザー情報 + ログアウト */ }
    <div className="border-t border-sidebar-border
p-4">
      <div className="mb-3 flex items-center gap-3">
        <div className="flex h-9 w-9 items-center
justify-center rounded-full bg-sidebar-accent text-sm
font-medium text-sidebar-accent-foreground">
          {session.user.name?.[0] || 'U'}
        </div>
        <div className="flex flex-col">
          <span className="text-sm font-medium
text-sidebar-foreground">
            {session.user.name}
          </span>
          <span className="text-xs
text-sidebar-foreground/60">
            {session.user.role === 'ADMIN'
              ? '管理者'
              : 'ユーザー'}
          </span>
        </div>
      </div>
    </div>

```

Day 08: サイドバー付きのアプリレイアウトを作ろう

ここはサイドバーの下に、今ログインしている人の情報を出す部分です。

`session.user.name?.[0]` は名前の 1 文字目だけを取り出し、丸いアイコンの中に入れます。`?.` は途中の値が無ければそこで止めて `undefined` を返す書き方で、名前が未設定でも画面が落ちません。取り出せなかったときは `|| 'U'` が働き、丸の中は `U` になります。

この `session` は 3-2 で書いた `getSession` の結果なのでログインした本人の名前と権限がそのまま出ます。別のユーザーでログインし直せばここの表示も入れ替わります。

確認ポイント:

- ☐ サイドバー下部にユーザー名と権限を表示しています。

```

    { /* filepath:
src/component/layout/app-layout.tsx(続き) */ }
    { /* ログアウトボタン(確認ダイアログ付き) */ }
    <AlertDialog
      open={logoutDialogOpen}
      onOpenChange={handleLogoutDialogOpenChange}
    >
      <AlertDialogTrigger asChild>
        <Button
          variant="outline"
          size="sm"
          className="w-full gap-2
border-sidebar-border text-sidebar-foreground/80
hover:bg-sidebar-accent hover:text-sidebar-foreground"
          onClick={openLogoutDialog}
        >
          <LogOut className="h-4 w-4" />
          ログアウト
        </Button>
      </AlertDialogTrigger>

```

ダイアログを自分で開閉するため、`open` に状態を渡します。

`AlertDialogTrigger` は開いたボタンを覚え、閉じたときにキーボードのフォーカスを戻します。`openLogoutDialog` は前回のエラー文を消してから開きます。

Day 08: サイドバー付きのアプリレイアウトを作ろう

ログアウトをすぐ実行せずダイアログを挟むのは押し間違いで作業中の画面から追いつ出されるのを防ぐためです。Day 07 で作ったログアウトは呼べばその場でセッションが消えます。取り消せない操作の手前に一段置く、という考え方はこの先の削除機能でも同じです。

確認ポイント:

- ☐ ボタンを押すと確認ダイアログが開きます。

```
    { /* filepath:
src/component/layout/app-layout.tsx(続き) */ }

    <AlertDialogContent>
      <AlertDialogHeader>
        <AlertDialogTitle>
ログアウトしますか？</AlertDialogTitle>
        <AlertDialogDescription>
          ログアウトすると、再度ログインが必要になります。
        </AlertDialogDescription>
      </AlertDialogHeader>
      {logoutError ? <p role="alert">{
logoutError}</p> : null}
      <AlertDialogFooter>
        <AlertDialogCancel disabled={
logoutMutation.isPending}>
          キャンセル
        </AlertDialogCancel>
        <Button onClick={handleLogout} disabled={
logoutMutation.isPending}>
          {logoutMutation.isPending
```

```

        ? 'ログアウト中...'
        : logoutError
        ? 'もう一度試す'
        : 'ログアウト'}

    </Button>

  </AlertDialogFooter>

</AlertDialogContent>

</AlertDialog>

</div>

</aside>

```

ダイアログの下部にはキャンセルと実行のボタンがあります。実行には普通の `Button` を使うため、通信に失敗してもダイアログは勝手に閉じません。送信中は両方を無効にし、失敗後は実行ボタンを「もう一度試す」に変えます。

ログアウトは Day 07 で `publicProcedure` として作り、Cookie の削除を繰り返しても同じ結果になる形です。先の応答だけ失われた場合も同じ削除を再試行でき、成功が返ったときだけキャッシュを消して `/login` へ移ります。

確認ポイント:

- ☐ ログアウト前に確認ダイアログを表示しています。

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
{/* filepath: src/component/layout/app-layout.tsx(続き)
*/}

{/* 画面に main は1つだけ置くため、各ページ側では書
かない */}

<main className="flex-1 overflow-y-auto p-6">
  {children}
</main>
</div>
);
}
```

AlertDialog の構造:

パーツ	役割
<code>open</code> / <code>onOpenChange</code>	ダイアログの開閉を状態で管理
<code>AlertDialogTrigger</code>	開くボタンを記録し、閉じた後にフォーカスを戻す
<code>AlertDialogContent</code>	ダイアログの中身
<code>Button</code>	ログアウトの実行と再試行
<code>AlertDialogCancel</code>	キャンセル(閉じるだけ)

実行ボタンを普通の `Button` にすると、失敗時にダイアログを開いたまま案内できます。

確認ポイント:

- ☐ `src/component/layout/app-layout.tsx` が作成できました。
- ☐ `'use client'` が先頭にあります。
- ☐ `menuItems` に「ダッシュボード」が1つだけあります。
- ☐ ログアウトボタンに `AlertDialog` が付いています。

学んだこと: `useQuery` でサーバーのセッション情報を取得し、無ければログイン画面へ送ります。レイアウト全体が認証ゲートの役割を持ちます。

Step 4: ダッシュボードに AppLayout を適用する(読む目安: 5分)

ゴール: Day 02 で作ったダッシュボードを、サイドバー付きの AppLayout で囲みます。

Day 09 以降も、各ページを同じ方法で AppLayout の中へ入れます。ファイルは移動せず、`src/app/dashboard/page.tsx` をそのまま編集します。

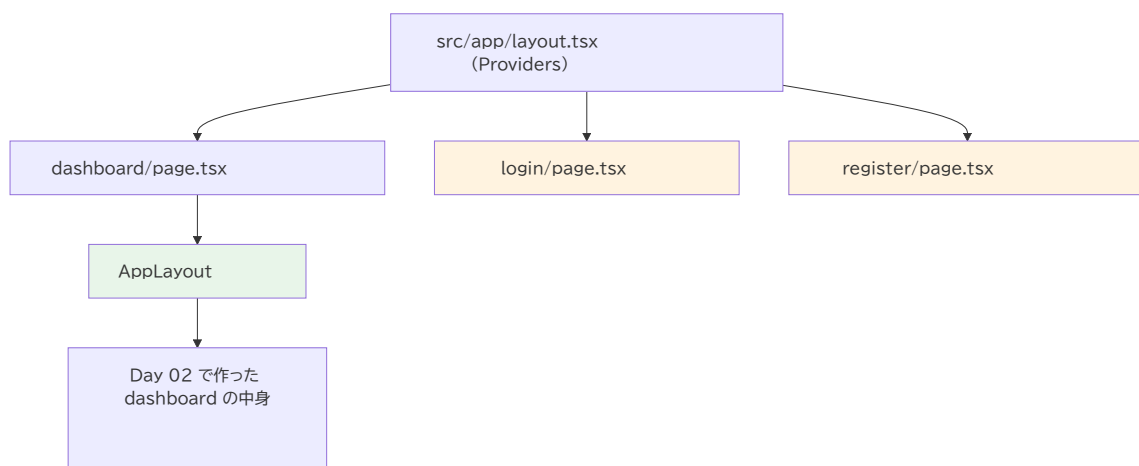


図6: Step 4: ダッシュボードに AppLayout を適用する

Day 08: サイドバー付きのアプリレイアウトを作ろう

図のいちばん上にある `src/app/layout.tsx` はすべてのページの親なのでログイン画面と登録画面もここを通ります。一方 `AppLayout` はダッシュボードの内側にだけ置きます。サイドバーを出したい画面と出たくない画面はこの置き場所だけで分かります。

ページを増やすときも同じで、サイドバーが要るページだけ `AppLayout` で囲みます。ログイン画面を囲んでしまうとログインしていない人が認証チェックに引っかかり、ログイン画面から `/login` へ送られ続けます。

`src/app/dashboard/page.tsx` の import に `AppLayout` を追加します。

```
// filepath: src/app/dashboard/page.tsx(import に追加)
import { AppLayout } from '@component/layout/app-layout';
```

次に Day 02 で作った外側の `<main>...</main>` を `<div>...</div>` に変え、その全体を `<AppLayout>...</AppLayout>` で囲みます。AppLayout 側が `<main>` を持つため内側は `<div>` にして重複を避けます。中身は消さず、そのまま残してください。

```
// filepath: src/app/dashboard/page.tsx(return の外側を変更)
return (
  <AppLayout>
    <div className="min-h-screen bg-background
text-foreground">
      {/* Day 02 で作った内容は、そのままここに残す */}
    </div>
  </AppLayout>
);
```

AppLayout がサイドバーと認証チェックを担当し、内側の `<div>` は Day 02 のダッシュボード表示を担当します。ログイン画面と登録画面は AppLayout で囲まれないため認証前でもサイドバーなしで表示されます。

確認ポイント:

- ☐ `src/app/dashboard/page.tsx` に AppLayout の import があります。
- ☐ `return` の外側が `<AppLayout>...</AppLayout>` で囲まれています。
- ☐ Day 02 の外側の `<main>` を `<div>` に変え、中身が残っています。

Step 5: ログインして全体の動作を確認する(読む目安: 5分)

ゴール: ここまでの全 Step が正しく連携して動くことを確認します。

```
npm run dev
```

Day 08: サイドバー付きのアプリレイアウトを作ろう

ここで一度ログアウトした状態に戻します。Day 07 でログインしたときの Cookie (ブラウザに保存された合言葉)は7日間有効なので続けて進めた人はまだログイン済みです。そのままだと手順1のリダイレクトが起きません。

ブラウザのシークレットウィンドウ(Cookie を持ち込まない別ウィンドウ)を開いてください。Chrome なら Mac は **Command + Shift + N**、Windows は **Ctrl + Shift + N** です。以降の確認はこのウィンドウで行います。

シークレットウィンドウで **http://localhost:3000** を開きます。

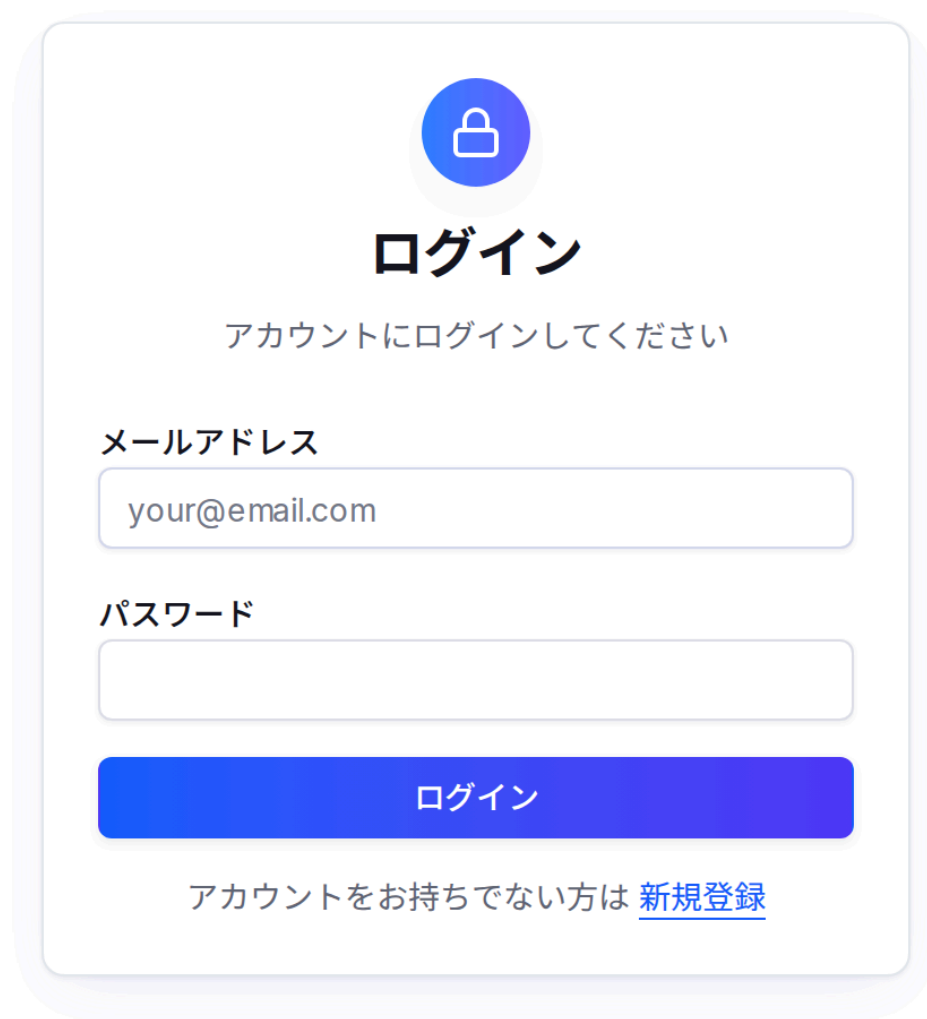
A login form with a blue circular icon containing a white padlock. Below the icon is the title 'ログイン' (Login) in bold black text, followed by the instruction 'アカウントにログインしてください' (Please login to your account). There are two input fields: 'メールアドレス' (Email address) with the placeholder 'your@email.com' and 'パスワード' (Password). A blue button labeled 'ログイン' (Login) is positioned below the password field. At the bottom, there is a link '新規登録' (Sign up) in blue text, preceded by the text 'アカウントをお持ちでない方は' (If you do not have an account).

図 7: ログイン画面。メールアドレスとパスワードの入力欄、ログインボタン、登録リンクが並んでいる

確認フロー:

1. `/dashboard` へアクセス → middleware が `/login` にリダイレクト。
2. `admin@example.com` / `password123` でログイン。
3. ダッシュボードが表示されます(サイドバー付き)。

Day 08: サイドバー付きのアプリレイアウトを作ろう



図8: 左に「ダッシュボード」だけのサイドバー、右に Day 02 で作ったダッシュボードのカードが並んでいる画面

画像と手元の画面で、サイドバーが「ダッシュボード」の1項目になっていることを確認します。まだ作っていないページへのメニューは表示されません。

4. サイドバーに「ダッシュボード」のメニューが見えます。
5. 「ダッシュボード」を押すと `/dashboard` が開き、404 になりません。
6. サイドバー下部に「管理者」の名前とロールが表示されます。
7. 「ログアウト」ボタンを押す → 確認ダイアログが出ます。

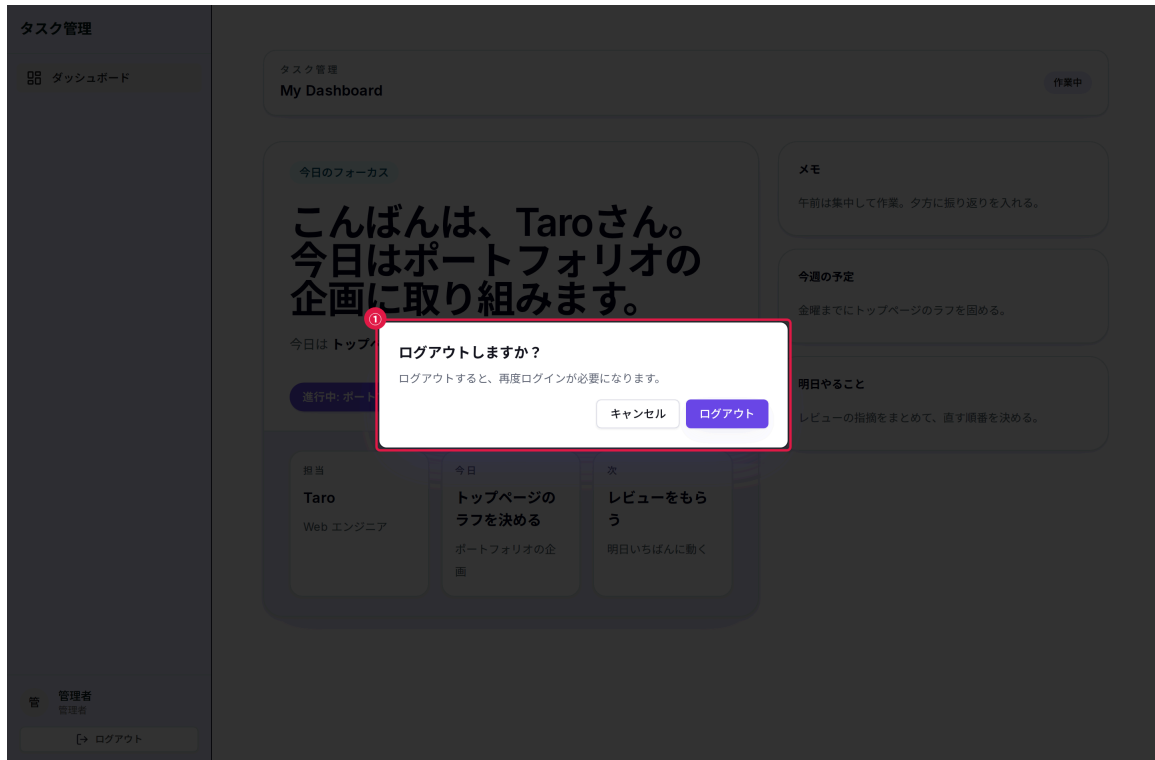


図9: 「ログアウトしますか？」の見出しと「ログアウトすると再度ログインが必要になります。」の説明、キャンセルとログアウトの2つのボタンが並んだ確認ダイアログ

8. 「ログアウト」を押す → `/login` に戻ります。

確認ポイント:

- ☐ `npm run dev` でエラーが出ません。
- ☐ サイドバーが左側に表示されます。
- ☐ ユーザー名「管理者」とロール「管理者」が見えます。
- ☐ ナビゲーションのアクティブ状態が正しいです(現在のページがハイライト)。
- ☐ ログアウト → 確認ダイアログ → ログイン画面に戻ります。

うまくいかないとき: 「つまずきポイント」セクションを確認してください。

Pro パターンで書こう(`use client` の影響範囲を最小化する)

Step 3 では `AppLayout` 全体を `'use client'` にしました。これは今日のスコープでは正しい書き方ですが次のステップでさらに改善できます。

Before(今日書いたコード・動くけど改善の余地あり)

```
// app-layout.tsx 全体が 'use client'
// → ナビゲーションリンク(静的)まで Client Component になる
```

今日書いた `app-layout.tsx` は先頭に `'use client'` を置いたため、ナビゲーションの構成やロゴの描画もブラウザ側で扱います。動きは正しく、最初に書く形としてはこれで問題ありません。

分け方の例(今日は実装しません)

次は構成を考えるための例です。今日書いたコードは置き換えません。

```
// Server Component から静的な外枠と Client の部品を組み立てる
// 認証判定・現在地の表示・ログアウトなどを Client Component に置く
// Client のページから外枠を import したままでは Server へ変わらない
```

現在の `AppLayout` は、認証判定や画面状態の管理に `useQuery`、`useState`、`useEffect` などを使います。ログアウトボタンだけをも移しても、これらの処理が残るため `'use client'` を外せません。

静的な外枠を Server Component にするなら、Hooks(状態の管理などに使う React の関数)を使う部分を Client Component へ分け、Server 側から組み立てます。外枠を Client のページから直接 `import` すると、その外枠も Client 側として扱われます。呼び出す側も含めて分ける必要があります。

ロゴなどの静的な部分をサーバー側に残せると、ブラウザへ送るコードを減らせます。ただし、初期表示が速くなるかは変更後の測定で確かめます。

覚えておきたいエッセンス: `'use client'` はブラウザで動かす処理の入口を示す宣言です。そのファイルだけでなく、そこから読み込む処理も Client 側へ含まれます。今日は `AppLayout` 全体を Client のまま使います。この節では、分割の実装は行いません。

完成コード全体

今日は4つのファイルを扱いました。このうち `providers.tsx` と `layout.tsx` は中身の確認だけです。断片を貼り重ねる作業が続いたので途中でどこへ貼ったか分からなくなった場合は以下のコードと自分のファイルを見比べてください。上から順に読めば Step 1 から Step 4 で扱ったものがどう1つのファイルになったかを確かめられます。

ファイル	役割	対応する Step
<code>src/app/providers.tsx</code>	tRPC と通知の 入口をアプリ全体 へ配る	Step 1

Day 08: サイドバー付きのアプリレイアウトを作ろう

ファイル	役割	対応する Step
<code>src/app/layout.tsx</code>	全ページを Provider で囲 む土台	Step 2
<code>src/component/layout/app-layout.tsx</code>	サイドバーとログ イン判定	Step 3
<code>src/app/dashboard/page.tsx</code>	AppLayout で 囲んだダッシュ ボード	Step 4
<code>src/app/providers.tsx</code>		

Provider の本体:

```
// filepath: src/app/providers.tsx
// 完成版: Provider の本体
'use client';

import { Toaster } from 'react-hot-toast';
import { TRPCReactProvider } from '@trpc/react';

export function Providers({ children }: { children:
React.ReactNode }) {
  return (
    <TRPCReactProvider>
      {children}
      <Toaster />
    </TRPCReactProvider>
  );
}
```

このファイルは Day 01 の scaffold が配布したままなので手元と1文字も違わないはずです。`TRPCReactProvider` で囲んだ内側だけが `api` を使えます。囲みの外に置いた部品から呼ぶと送り先が決まらずエラーになります。`<Toaster />` を `{children}` の隣に1つだけ置いているのは通知の描画先をアプリ全体で1か所にそろえるためです。ページごとに置くと同じ通知が置いた枚数だけ重なって出ます。

`src/app/layout.tsx`

取り込みとフォントの読み込み:

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
// filepath: src/app/layout.tsx
// 完成版: 取り込みとフォントの読み込み
import type { Metadata } from 'next';
import { Inter, JetBrains_Mono, Noto_Sans_JP } from
'next/font/google';
import { Providers } from './providers';
import './globals.css';

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-inter',
  display: 'swap',
});

const jetBrainsMono = JetBrains_Mono({
  subsets: ['latin'],
  variable: '--font-jetbrains-mono',
  display: 'swap',
});

const notoSansJP = Noto_Sans_JP({
  subsets: ['latin'],
  weight: ['400', '500', '700'],
  variable: '--font-noto-sans-jp',
  display: 'swap',
});
```

Step 2 では `Inter` の1つ分だけを載せました。残る2つも書き方は変わりません。3つとも関数の外側で呼んでいるのは `next/font/google` がフォントの取り寄せをビルドのときに1回だけ済ませる作りだからです。関数の中へ入れると画面を描き直すたびに同じ指定を組み立て直すことになります。`variable` で CSS の変数名を決めておくと `globals.css` 側はこの名前だけを見れば済みます。

題名と全ページの外枠:

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
// filepath: src/app/layout.tsx(同じファイルの続き)

// 完成版: 題名と全ページの外枠

export const metadata: Metadata = {
  title: 'TaskApp - プロジェクト・タスク管理',
  description:
    'チームで使えるプロジェクト・タスク管理アプリケーション。'
  +
    'プロジェクト管理、タスクトラッキング、進捗レポート機能を提供します。',
};

export default function RootLayout({ children }: {
children: React.ReactNode }) {
  return (
    <html
      lang="ja"
      className={` ${inter.variable} ${
jetBrainsMono.variable} ${notoSansJP.variable}`}
    >
      <body>
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}
```

`metadata` は Step 2 の抜粋には出てきませんが scaffold が配った時点ですでに入っています。ブラウザのタブに出る題名と、検索結果に出る説明文がここで決まります。

`<Providers>` を `<body>` のすぐ内側に置いてあるのはこの位置より内側にある全ページへ tRPC を届けるためです。フォントの変数を `<html>` に付けているのも理由は揃っていていちばん外側に置けば配下のどのページからも読めます。

`src/component/layout/app-layout.tsx`

ブラウザ側の宣言と外部ライブラリの取り込み:

```
// filepath: src/component/layout/app-layout.tsx
// 完成版: ブラウザ側の宣言と外部ライブラリの取り込み
'use client';

import { useQueryClient } from '@tanstack/react-query';
import { LayoutDashboard, Logout } from 'lucide-react';
import Link from 'next/link';
import { usePathname, useRouter } from 'next/navigation';
import { useEffect, useRef, useState } from 'react';
```

1 枚目は外部ライブラリの取り込みです。並びがアルファベット順になっているのは `npm run fix` を実行すると Biome(このプロジェクトのコード整形ツール)が並べ替えるからです。自分が書いた順番と違っていても手で直す必要はありません。`'use client'` はファイルの1行目に置きます。取り込みの後ろへ動かすとブラウザ側で動かす宣言として読まれず、`useState` の行でエラーになります。

画面部品と tRPC の取り込み:

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
// filepath: src/component/layout/app-layout.tsx(同じファイルの続き)
// 完成版: 画面部品と trpc の取り込み

import {
  AlertDialog,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from '@component/ui/alert-dialog';
import { Button } from '@component/ui/button';
import { cn } from '@lib/utils';
import { api } from '@trpc/react';
```

2枚目は自分のプロジェクトの中にある部品です。`@/` で始まる書き方は `src/` を指す近道で、ファイルの置き場所が深くなっても `../..` を数えずに済みます。

`AlertDialog` で始まる名前が8個並ぶのはshadcn/uiのダイアログが枠・見出し・本文・ボタンに分かれているからです。手元で数えて8個に足りない場合はその足りない名前を書いた行が「定義されていません」というエラーになります。

メニュー項目の型:

```
// filepath: src/component/layout/app-layout.tsx(同じファイルの続き)
// 完成版: メニュー項目の型
interface MenuItem {
  text: string;
  icon: React.ReactNode;
  path: string;
}
```

メニュー1件の形をここで決めておくと項目を足すときに何を書けばよいかが決まります。`icon` の型が `React.ReactNode` になっているのは文字列ではなく `<LayoutDashboard />` のような画面部品をそのまま入れるからです。

メニュー項目の一覧:

```
// filepath: src/component/layout/app-layout.tsx(同じファイルの続き)
// 完成版: メニュー項目の一覧
const menuItems: MenuItem[] = [
  {
    text: 'ダッシュボード',
    icon: <LayoutDashboard className="h-5 w-5" />,
    path: '/dashboard',
  },
];
```

この配列を関数の外に置いてあるのは中身が画面の状態が変わらないからです。関数の中へ入れると画面を描き直すたびに同じ1件を作り直します。並びはそのまま画面の上から下の順になります。項目を増やすときに書き足すのはこの配列だけで、後ろに出てくる表示側は書き換えません。

Day 08: サイドバー付きのアプリレイアウトを作ろう

状態とサーバーへの問い合わせ:

```
// filepath: src/component/layout/app-layout.tsx(同じファイルの続き)

// 完成版: 状態とサーバーへの問い合わせ

export function AppLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const [hasMounted, setHasMounted] = useState(false);

  const [logoutDialogOpen, setLogoutDialogOpen] =
    useState(false);

  const [logoutError, setLogoutError] = useState<string |
    null>(null);

  const logoutLockedRef = useRef(false);

  const queryClient = useQueryClient();

  const pathname = usePathname();

  const router = useRouter();

  const { data: session, isLoading } =
    api.auth.getSession.useQuery();
```

ログアウトの失敗を画面に残すため、ダイアログの開閉とエラー文を `useState` で覚えます。 `useRef` (再描画を待たずに値を保持する React の仕組み)は、連続クリックをその場で止めるために使います。

```
// filepath: src/component/layout/app-layout.tsx(続き)
const logoutMutation = api.auth.logout.useMutation({
  onSuccess: async () => {
    await queryClient.cancelQueries();
    queryClient.clear();
    router.push('/login');
    router.refresh();
    setLogoutDialogOpen(false);
  },
  onError: () => {
    setLogoutError(
      'ログアウトの完了を確認できませんでした。' +
      '通信状況を確認して、もう一度お試しください。');
  },
  onSettled: () => {
    logoutLockedRef.current = false;
  },
});

const handleLogout = () => {
  if (logoutLockedRef.current ||
logoutMutation.isPending) return;
  logoutLockedRef.current = true;
  setLogoutError(null);
  logoutMutation.mutate();
};
```

Day 08: サイドバー付きのアプリレイアウトを作ろう

`onSettled` (成功・失敗のどちらでも通信の最後に動く処理)で

`logoutLockedRef` のロックを解除します。失敗した後も解除されるため、案内を読んでから再試行できます。

```
// filepath: src/component/layout/app-layout.tsx(同じファイルの続き)
// 完成版: ログアウトの操作

const openLogoutDialog = () => {
  setLogoutError(null);
  setLogoutDialogOpen(true);
};

const handleLogoutDialogOpenChange = (open: boolean) =>
{
  if (!open && (logoutLockedRef.current ||
logoutMutation.isPending)) return;

  if (open) setLogoutError(null);
  setLogoutDialogOpen(open);
};
```

関数の先頭が道具をそろえる場所です。`useState` や `useQueryClient`、`useQuery` は毎回同じ順番で呼ばれることを前提に作られているので `if` の中や `return` の後ろへ動かすと React が値を取り違えます。`logoutMutation` をここで用意しておくのも同じ理由です。実際に通信が始まるのは後ろのボタンから `mutate()` が呼ばれたときで、この行では準備だけをしています。

成功時は `cancelQueries()`、`clear()`、画面移動の順です。先に問い合わせを止めるため、ログアウト直前に始まった取得処理が古いデータをキャッシュへ戻すことも防げます。失敗時はキャッシュを消さず、同じダイアログから再試行できます。

未ログインの人を送り出す判定:

```
// filepath: src/component/layout/app-layout.tsx(同じファイルの続き)

// 完成版: 未ログインの人を送り出す判定

useEffect(() => {
  setHasMounted(true);
}, []);

useEffect(() => {
  if (hasMounted && !isLoading && !session) {
    router.push('/login');
  }
}, [hasMounted, isLoading, session, router]);

if (!hasMounted || isLoading) {
  return (
    <div className="flex h-screen items-center justify-center">
      <p className="text-muted-foreground">
        読み込み中...
      </p>
    </div>
  );
}

if (!session?.user) {
  return null;
}
```

Day 08: サイドバー付きのアプリレイアウトを作ろう

ここが今日のいちばんの要点です。2つの `if` を `return` より前に置いてあるのでこの行より下は「ログイン済みの人だけが通る場所」になります。だから後ろのサイドバーでは `session?.user?.name` と書かずに `session.user.name` と書けます。ただし名前そのものは未設定があり得るので頭文字を取り出す `?.[0] || 'U'` は残します。判定を後ろへ動かすと未ログインの人にサイドバーが一瞬見えてしまいます。`return null` は何も描かずに終わる書き方で、`/login` への移動が終わるまでの短い間だけ働きます。

サイドバーの外枠とロゴ:

```
// filepath: src/component/layout/app-layout.tsx(同じファイルの続き)
// 完成版: サイドバーの外枠とロゴ

return (
  <div className="flex h-screen bg-background">
    {/* サイドバー */}
    <aside className="hidden w-64 flex-col border-r
bg-sidebar md:flex">
      {/* ロゴ */}
      <div className="border-b border-sidebar-border
p-4">
        <h1 className="text-lg font-bold
text-sidebar-foreground">
          タスク管理
        </h1>
      </div>
    </div>
  </div>
)
```

いちばん外側を `flex` にしてあるのでサイドバーと本文が左右に並びます。

`h-screen` は画面の高さいっぱいを指し、これが無いとサイドバーの色帯が中身の高さで止まります。`<aside>` を使っているのは読み上げソフトに「ここは本文ではなく脇の案内です」と伝えるためです。見た目だけなら `<div>` でも同じに見えます。

ナビゲーション:

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
    { /* filepath:
src/component/layout/app-layout.tsx(同じファイルの続き)
*/ }

    { /* 完成版: ナビゲーション */ }
    { /* ナビゲーション */ }

    <nav className="flex-1 p-3">
      <ul className="space-y-1">
        {menuItems.map((item) => (
          <li key={item.path}>
            <Link
              href={item.path}
              className={cn(

'flex items-center gap-3 rounded-md px-3 py-2 text-sm ' +
              'transition-colors',
              pathname === item.path
                ?
'bg-sidebar-accent text-sidebar-accent-foreground' +
                ' font-medium'
                :
'text-sidebar-foreground/70 hover:bg-sidebar-accent/50' +
                ' hover:text-sidebar-foreground',
            )}
          </li>
        )}
      </ul>
    </nav>
```

この前半でリンク先と、現在地に応じた色を決めています。続きでアイコンと文字を描き、一覧のタグを閉じます。2つのブロックは同じ `Link` の続きなので、間に別のコードは入れません。

```

    >
      { /* filepath:
src/component/layout/app-layout.tsx(同じファイルの続き)
*/ }

      { /* 完成版: ナビゲーションの続き */ }
      {item.icon}
      {item.text}
    </Link>
  </li>
)}}
</ul>
</nav>

```

`<nav>` に付けた `flex-1` が余った高さをこの区画に全部渡します。だから下のユーザー情報はメニューの件数が変わってもサイドバーの底に貼り付きます。`cn` に渡す2つ目の値は開いているページかどうかで切り替わります。この切り替えが無いとユーザーは自分がどこにいるかを画面から判断できません。

ユーザー情報:

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
    { /* filepath:
src/component/layout/app-layout.tsx(同じファイルの続き)
*/ }

    { /* 完成版: ユーザー情報 */ }
    { /* ユーザー情報 + ログアウト */ }

    <div className="border-t border-sidebar-border
p-4">

        <div className="mb-3 flex items-center gap-3">

            <div className="flex h-9 w-9 items-center
justify-center rounded-full bg-sidebar-accent text-sm
font-medium text-sidebar-accent-foreground">

                {session.user.name?.[0] || 'U'}

            </div>

            <div className="flex flex-col">

                <span className="text-sm font-medium
text-sidebar-foreground">

                    {session.user.name}

                </span>

                <span className="text-xs
text-sidebar-foreground/60">

                    {session.user.role === 'ADMIN'
                    ? '管理者'
                    : 'ユーザー'}

                </span>

            </div>

        </div>

    </div>
```

丸の中へ `session.user.name?.[0] || 'U'` を入れています。名前が未設定の人でも、丸を空にしないためです。`?.` は途中の値が無ければそこで止める書き方です。取り出せなかったときは `|| 'U'` が働きます。ここに出る名前は手前で受け取った `session` の中身そのものです。別のユーザーとしてログインし直せばこの表示も入れ替わります。

ログアウトボタン:

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
    { /* filepath:
src/component/layout/app-layout.tsx(同じファイルの続き)
*/ }

    { /* 完成版: ログアウトボタン */ }
    { /* ログアウトボタン(確認ダイアログ付き) */ }
    <AlertDialog
      open={logoutDialogOpen}
      onOpenChange={handleLogoutDialogOpenChange}
    >
      <AlertDialogTrigger asChild>
        <Button
          variant="outline"
          size="sm"
          className="w-full gap-2
border-sidebar-border text-sidebar-foreground/80
hover:bg-sidebar-accent hover:text-sidebar-foreground"
          onClick={openLogoutDialog}
        >
          <LogOut className="h-4 w-4" />
          ログアウト
        </Button>
      </AlertDialogTrigger>
```

`open` と `onOpenChange` を渡すと、ダイアログの開閉を `logoutDialogOpen` で管理できます。`AlertDialogTrigger` は閉じた後のフォーカスをログアウトボタンへ戻します。送信中は閉じる操作を受け付けず、失敗したときは案内をそのまま読めます。

ログアウトの確認ダイアログ:

```

    { /* filepath:
src/component/layout/app-layout.tsx(同じファイルの続き)
 */

    <AlertDialogContent>
      <AlertDialogHeader>
        <AlertDialogTitle>
ログアウトしますか？</AlertDialogTitle>
        <AlertDialogDescription>
          ログアウトすると、再度ログインが必要になります。
        </AlertDialogDescription>
      </AlertDialogHeader>
      {logoutError ? <p role="alert">{
logoutError}</p> : null}
      <AlertDialogFooter>
        <AlertDialogCancel disabled={
logoutMutation.isPending}>
          キャンセル
        </AlertDialogCancel>
        <Button onClick={handleLogout} disabled={
logoutMutation.isPending}>
          {logoutMutation.isPending
            ? 'ログアウト中...'
            : logoutError
              ? 'もう一度試す'
              : 'ログアウト'}
        </Button>
      </AlertDialogFooter>
    </AlertDialogContent>
  )}

```

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
        </AlertDialogContent>
      </AlertDialog>
    </div>
  </aside>
```

実行には普通の `Button` を使います。`AlertDialogAction` は押すと閉じるため、通信失敗を表示したい場面には向きません。`useRef` のロックは React の再描画より先に更新され、素早い連続クリックでも要求を1回に抑えます。最後の `</aside>` でサイドバー全体を閉じています。

本文の置き場所:

```
    { /* filepath: src/component/layout/app-layout.tsx(
同じファイルの続き) */ }

    { /* 完成版: 本文の置き場所 */ }

    { /* 画面に main は1つだけ置くため、各ページ側では書
かない */ }

    <main className="flex-1 overflow-y-auto p-6">
      {children}
    </main>
  </div>
);
}
```

`{children}` は、この枠で囲んだページの中身が入る場所です。

`overflow-y-auto` を本文側だけに付けてあるので縦に長いページでもサイドバーは動かず、本文だけがスクロールします。ページ側が `<main>` を持たないのはこの行がすでに1つ持っているからです。

src/app/dashboard/page.tsx

Step 4 で書き換えたのは取り込みの1行と `return` の外枠だけです。中の表示は Day 02 で書いたものがそのまま残ります。名前や文言を自分用に書き換えている場合はその値のまま残してください。以下は Day 02 で書いた内容を残したまま外側を囲み直した状態です。

取り込みと型定義:

```
// filepath: src/app/dashboard/page.tsx
// 完成版: 取り込みと型定義
import { AppLayout } from '@component/layout/app-layout';

type DashboardOwner = {
  name: string;
  role: string;
  todayFocus: string;
  todayGoal: string;
  nextAction: string;
};

type FocusCard = {
  label: string;
  value: string;
  description: string;
};
```

Day 08: サイドバー付きのアプリレイアウトを作ろう

Day 02 のこのファイルには取り込みが1行もありませんでした。今日ここに

`AppLayout` の1行が増えます。型を2つ置いてあるのは Day 02 のままです。型はブラウザへ送られる前に消えるので増やしても読み込みは重くなりません。

表示する値とあいさつの組み立て:

```
// filepath: src/app/dashboard/page.tsx(同じファイルの続き)
// 完成版: 表示する値とあいさつの組み立て

const dashboardOwner: DashboardOwner = {
  name: 'Taro',
  role: 'Web エンジニア',
  todayFocus: 'ポートフォリオの企画',
  todayGoal: 'トップページのラフを決める',
  nextAction: 'レビューをもらう',
};

function getGreetingByHour(hour: number): string {
  if (hour < 12) {
    return 'おはよう';
  }

  if (hour < 18) {
    return 'こんにちは';
  }

  return 'こんばんは';
}
```

`dashboardOwner` は人によって変わる値を1か所へ集めた入れ物です。名前を自分のもの書き換えた人はここが違って正解です。`getGreetingByHour` を関数に分けてあるのは時刻の判定を表示側から切り離すためです。この関数は時刻だけを受け取り、画面のことを知りません。

画面の関数とカードの並び:

```
// filepath: src/app/dashboard/page.tsx(同じファイルの続き)
// 完成版: 画面の関数とカードの並び

function buildMainMessage(owner: DashboardOwner, hour:
number): string {
  const greeting = getGreetingByHour(hour);

  return `${greeting}、${owner.name}さん。今日は${
owner.todayFocus}に取り組みます。`;
}

export default function DashboardPage() {
  const currentHour = new Date().getHours();
  const mainMessage = buildMainMessage(dashboardOwner,
currentHour);
  const focusCards: FocusCard[] = [
    {
      label: '担当',
      value: dashboardOwner.name,
      description: dashboardOwner.role,
    },
  ],
```

Day 08: サイドバー付きのアプリレイアウトを作ろう

`focusCards` の中で `dashboardOwner.name` を読んでいるのは名前をもう一度手で書かないためです。手で書くと `dashboardOwner` を直したのに下段のカードだけ古い名前が残ります。この配列の並び順がそのまま画面の左から右の並びになります。

残りのカードと外枠の開始:

```
// filepath: src/app/dashboard/page.tsx(同じファイルの続き)
// 完成版: 残りのカードと外枠の開始

    {
      label: '今日',
      value: dashboardOwner.todayGoal,
      description: dashboardOwner.todayFocus,
    },
    {
      label: '次',
      value: dashboardOwner.nextAction,
      description: '明日いちばんに動く',
    },
  ];

  return (
    <AppLayout>
      <div className="min-h-screen bg-background
text-foreground">
        <div className="mx-auto flex min-h-screen
max-w-6xl flex-col px-6 py-8 lg:px-10">
```

今日の書き換えはここです。Day 02 では `return (` のすぐ後ろが

`<main className="min-h-screen ..." >` でした。それを `<AppLayout>`

で囲み、内側は `<div>` に変えています。`<main>` を残したままにすると

`AppLayout` が持つ `<main>` と2つ重なります。読み上げソフトは本文の入れ物を2つ見つけることになり、どちらが本文なのか決められません。

ヘッダー:

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
    { /* filepath: src/app/dashboard/page.tsx(同じフ
    アイルの続き) */ }

    { /* 完成版: ヘッダー */ }

    <header className="flex flex-col gap-3
rounded-2xl border border-border bg-card px-5 py-4
shadow-sm sm:flex-row sm:items-center sm:justify-between">

      <div className="space-y-1">

        <p className="text-xs font-medium uppercase
tracking-[0.24em] text-muted-foreground">
          タスク管理
        </p>

        <h1 className="text-lg font-semibold
text-card-foreground">
          My Dashboard
        </h1>
      </div>

      <div className="inline-flex w-fit
items-center rounded-full bg-secondary px-3 py-1.5
text-xs font-medium text-secondary-foreground">
        作業中
      </div>
    </header>
```

中身は Day 02 のままです。囲みが1つ増えたぶん、字下げが2つ分だけ深くなっています。写経し直す必要はありません。エディタの整形機能を使えば囲みを足した時点で自動的にこの位置へ揃います。

主役のメッセージ:

```

    {/* filepath: src/app/dashboard/page.tsx(同じフ
    アイルの続き) */}

    {/* 完成版: 主役のメッセージ */}

    <section className="mt-8 grid gap-6
    lg:grid-cols-[1.2fr_0.8fr]">

        <article className="overflow-hidden
        rounded-3xl border border-border bg-card shadow-md">

            <div className="border-b border-border px-8
            py-6">

                <span className="inline-flex items-center
                rounded-full bg-accent px-3 py-1 text-sm font-medium
                text-accent-foreground">

                    今日のフォーカス

                </span>

                <h2 className="mt-6 max-w-4xl text-4xl
                font-semibold tracking-tight text-card-foreground
                sm:text-5xl">

                    {mainMessage}

                </h2>

```

`{mainMessage}` に入るのは手前の `buildMainMessage` が組み立てた1本の文字列です。表示側は文の作り方を知らず、受け取って出すだけです。この分かれ方ができているとあいさつの言い回しを変えたいときに直すのは関数の側だけで済みます。

説明文と集中テーマの帯:

Day 08: サイドバー付きのアプリレイアウトを作ろう

```
        { /* filepath: src/app/dashboard/page.tsx(
同じファイルの続き) */}

        { /* 完成版: 説明文と集中テーマの帯 */}

        <p className="mt-4 max-w-2xl text-base
leading-8 text-muted-foreground">
            今日は
            <span className="font-semibold
text-foreground"> {dashboardOwner.todayGoal}</span>
            まで進めます。
        </p>

        <div className="mt-8 inline-flex
items-center rounded-full bg-primary px-4 py-2 text-sm
font-semibold text-primary-foreground shadow-sm">
            進行中: {dashboardOwner.todayFocus}
        </div>
    </div>
```

`max-w-2xl` や `max-w-4xl` で幅の上限を決めているのは文が長くなっても決まった位置で折り返すためです。上限が無いと画面の広い人だけ1行が端まで伸びて読みにくくなります。文字の色に `text-muted-foreground` を使っているので主役の見出しとの強弱が保たれます。

下段の3枚のカード:

```

      {/* filepath: src/app/dashboard/page.tsx(同
じファイルの続き) */}

      {/* 完成版: 下段の3枚のカード */}

      <div className="grid gap-4 bg-secondary
px-8 py-6 md:grid-cols-3">
        {focusCards.map((card) => (
          <article
            key={card.label}
            className="rounded-2xl border
border-border bg-background px-4 py-4 shadow-xs"
          >
            <p className="text-xs font-medium
uppercase tracking-[0.18em] text-muted-foreground">
              {card.label}
            </p>
            <p className="mt-3 text-lg
font-semibold text-foreground">
              {card.value}
            </p>
            <p className="mt-2 text-sm leading-7
text-muted-foreground">
              {card.description}
            </p>
          </article>
        ))}
      </div>
    </article>

```

Day 08: サイドバー付きのアプリレイアウトを作ろう

`focusCards` の要素が3つなのでカードも3枚出ます。読んでいるのは `label`・`value`・`description` の3項目で、`FocusCard` 型で決めた3つとそろっています。表示する項目を増やしたいときは型・配列・この中身の3か所を合わせて直します。

右側の補助カード:

```

    { /* filepath: src/app/dashboard/page.tsx(同じ
ファイルの続き) */}

    { /* 完成版: 右側の補助カード */}

    <aside className="space-y-4">

      <article className="rounded-3xl border
border-border bg-card p-6 shadow-sm">

        <p className="text-sm font-semibold
text-card-foreground">
          メモ
        </p>

        <p className="mt-4 text-sm leading-8
text-muted-foreground">
          午前は集中して作業。夕方に振り返りを入れる。
        </p>
      </article>

      <article className="rounded-3xl border
border-border bg-card p-6 shadow-sm">

        <p className="text-sm font-semibold
text-card-foreground">
          今週の予定
        </p>

        <p className="mt-4 text-sm leading-8
text-muted-foreground">
          金曜までにトップページのラフを固める。
        </p>
      </article>

```

Day 08: サイドバー付きのアプリレイアウトを作ろう

カードの間隔は親の `space-y-4` がまとめて決めています。だから枚数が変わっても1枚ずつ余白を書き足す作業は起きません。この `<aside>` は主役と同じ `<section>` の中にあるので画面が狭いときは主役の下へ回ります。

最後のカードと閉じタグ:

```
        { /* filepath: src/app/dashboard/page.tsx(同
        じファイルの続き) */}

        { /* 完成版: 最後のカードと閉じタグ */}

        <article className="rounded-3xl border
border-border bg-card p-6 shadow-sm">

            <p className="text-sm font-semibold
text-card-foreground">

                明日やること

            </p>

            <p className="mt-4 text-sm leading-8
text-muted-foreground">

                レビューの指摘をまとめて、直す順番を決める。

            </p>

        </article>

    </aside>

</section>

</div>

</div>

</AppLayout>

);

}
```

閉じタグが `</div>` から `</AppLayout>` へ続く形になっているかを、ここで確かめてください。Day 02 の終わりは `</main>` で閉じていました。今日はその外側に囲みが1つ増えたので閉じる順番も1段深くなります。ここが合っていればブラウザで `/dashboard` を開いたときに左のサイドバーと Day 02 の中身が同時に出来ます。

今日のまとめ

- ☐ `src/app/providers.tsx` — 配布済みの tRPC Provider の中身を確認しました。
- ☐ `src/app/layout.tsx` — `<Providers>` が組み込まれていることを確認しました。
- ☐ `src/component/layout/app-layout.tsx` — サイドバー付きレイアウトを作りました。
- ☐ `src/app/dashboard/page.tsx` — AppLayout で囲みました。
- ☐ ログイン → サイドバー表示 → ログアウトの一連の流れを確認しました。

つまずきポイント

`api is not defined`

原因

`@/trpc/react` からの import が漏れています。

解決方法

`import { api } from '@/trpc/react'` を確認します。

Day 08: サイドバー付きのアプリレイアウトを作ろう

`Cannot find module '@/component/ui/alert-dialog'`

原因

UI コンポーネントが配置されていません。

解決方法

`src/component/ui/alert-dialog.tsx` が存在するか確認します。

サイドバーが表示されない

原因

`dashboard/page.tsx` を `AppLayout` で囲んでいません。

解決方法

Step 4 の import と return を確認します。

ログイン後に白い画面

原因

`AppLayout` の `if (!session?.user) { return null; }` に入ったままです。セッションを取得できていません。

解決方法

Step 3 の `getSession` の呼び出しと、ログインが済んでいるかを確認します。

`useQuery` でエラー

原因

`auth` ルーターの登録、tRPC の Route Handler、または開発サーバーのどこかでエラーが発生しています。

解決方法

Day 07 の Step 4 に戻り、`src/server/api/root.ts` に `auth: authRouter` があることと、`src/app/api/trpc/[trpc]/route.ts` があることを確認してください。両方が合っている場合は、`npm run dev` を実行しているターミナルのいちばん上のエラーから確認してください。

ログアウトしてもリダイレクトされない

原因

`onSuccess` 内に `router.push('/login')` を書き忘れていました。

解決方法

`onSuccess` 内に `router.push('/login'); router.refresh();` の2行があるか確認します。`refresh()` だけを忘れた場合は、移動しても古いユーザー名が残ります。

今日学んだ用語

用語	意味
Provider	内側のすべての部品へ同じ値を配る囲み。どこに置くかで届く範囲が決まる
useQuery	サーバーからデータを取ってくる tRPC のフック

用語	意味
AppLayout	サイドバーとメインコンテンツに画面を分ける、自作の共通レイアウト
aside	「ここは本文ではなく脇の案内」という意味を持つ HTML のタグ
hasMounted	ブラウザ側で描画が始まったかを表す目印。サーバー側の結果とのちらつきを防ぐ
AlertDialog	取り消せない操作の前に確認を求める、shadcn/ui のダイアログ
asChild	ダイアログの引き金を、内側に書いた自分のボタンにそのまま任せる指定
md:	Tailwind CSS で「画面幅 768px 以上のときだけ効かせる」という接頭辞

応用課題: メニューの表示名とリンク先を見分ける

メニューの表示名だけを一時的に変え、リンク先が別の値として保たれていることを確かめます。

(1) `src/component/layout/app-layout.tsx` の `menuItems` を探し、ダッシュボードの要素を確認します。

(2) その要素の `text` だけを一時的に `ホーム` へ変えて保存します。

`path: '/dashboard'` は変えません。

(3) サイドバーが表示される広い画面で `/dashboard` を開き、表示だけが「ホーム」に変わることを確認します。確認後は `text` を `ダッシュボード` へ戻します。配列のデータとリンクの表示が対応していることを確かめる練習です。

(4) 表示名を戻したあと、「ダッシュボード」と表示されることを確認します。

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1. `AppLayout` の `if (!hasMounted || isLoading)` と `if (!session?.user) { return null; }` は何をしている2つですか。

A. 前者はセッションの問い合わせが終わるまで「読み込み中...」を出します。後者は問い合わせが終わってもログインしていなければ何も描かずに終わります。どちらも `return` で処理を打ち切ります。その結果、この2つより下の行は「ログイン済みの人だけが通る場所」になります。だからサイドバーでは `session?.user?.name` と書かずに `session.user.name` と書けます。ただし名前そのものは未設定があり得るので頭文字を取り出す `?.[0] || 'U'` は残します。

Q2. `menuItems` に4件目を足すと画面はどうなりますか。

A. サイドバーのメニューが4つになります。JSX の側は1行も書き足す必要がありません。`menuItems.map` が配列をそのまま回してリンクを作っているからです。並ぶ順番は配列に書いた順がそのまま上から下になります。

Q3. ダッシュボード側の外側の `<main>` を `<div>` に書き換えるのはなぜですか。

A. `AppLayout` がすでに `<main>` を1つ持っているからです。そのまま残すと本文の入れ物がページに2つ並びます。`<main>` は「ここがこのページの本文だ」という意味を持つタグです。2つあると読み上げソフトはどちらを本文と判断すればよいが決められません。

次回予告

Day 09 ではプロジェクト一覧画面を作ります。tRPC の `useQuery` でサーバーからプロジェクト一覧を取得してカードコンポーネントで表示します。今日作ったレイアウトの中に、最初の「データを表示する画面」が入ります。

次に読むもの

- 前の日: Day 07^{*1}
- 次の日: Day 09^{*2}
- 全体の地図: 学びのロードマップ^{*3}
- 目次: カリキュラム目次^{*4}
- 詰まったとき: トラブルシューティング^{*5}
- 言葉の意味: 用語集^{*6}

1. <https://drive.google.com/file/d/13-JuMAvw7EsJJJMYgyk5HayoeBiGqkTk/view?usp=drivesdk>

2. <https://drive.google.com/file/d/1DXLAWz2qvGhQeFEbi3RdLKQgd6L3foRt/view?usp=drivesdk>

3. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk

4. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>

5. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>

6. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>

奥付

タイトル	Day 08: サイドバー付きのアプリレイアウトを作ろう
著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。